Федеральное государственное образовательное бюджетное учреждение высшего образования
«Финансовый университет при Правительстве Российской Федерации»

Факультет «Информационных технологий и анализа больших данных»
Направление «Программная инженерия»

Практическая работа №2 «Знакомство с инструментальными средствами разработки кросс-платформенных программных продуктов»

По дисциплине «Кросс-платформенная разработка»

Выполнила: студентка 4 курса,
группы ТРПО23-1
очной формы обучения
Клефас Мария Сергеевна
Преподаватель:
кандидат педагогических наук, доцент
Бузыкова Юлия Сергеевна

Москва 2026

Ссылка на гитхаб: https://github.com/marieklefas/crossplatform-practice-02

1. Цель и задачи работы

Цель работы — сформировать системное представление о современных инструментальных средствах разработки кросс-платформенных программных продуктов и получить первичный практический опыт их применения.

В работе выполняются классификация инструментальных средств, подготовка рабочего окружения, анализ устройства готового кросс-платформенного проекта и сравнительный анализ технологий по индивидуальному варианту 6.

2. Задание 1. Классификация инструментальных средств

Таблица 1. Классификационная таблица инструментальных средст кросс-платформенной разработки

Категория	Инструмент	Разработчик	Язык	Целевые платформы	Механизм кросс-платформенности	Лицензия	Актуальная версия и дата	Пример применения
Интерпретируемые языки и среды	Python	Python Software Foundation	Python	Windows, macOS, Linux и др.	Интерпретатор CPython для каждой платформы	PSF License	3.15.1 / 21.10.2026	Dropbox, Blender-скрипты
Интерпретируемые языки и среды	Node.js	OpenJS Foundation	JavaScript	Windows, macOS, Linux	JavaScript-рантайм V8 для разных ОС	MIT	24.22.1 LTS / 21.10.2026	Инфраструктура VS Code, веб-сервисы
Виртуальные машины и байт-код	.NET / CLR	Microsoft	C#, F#, VB	Windows, Linux, macOS; мобильные через MAUI	IL-байт-код + CLR/JIT/AOT на целевой платформе	MIT (.NET)	.NET 10 LTS / 11.11.2025	PowerShell, .NET MAUI-приложения
Переносимый исходный код	CMake	Kitware	C/C++ и др.	Windows, macOS, Linux и др.	Генерация платформенных	BSD-3-Clause	4.4.4 / 08.10.2026	LLVM, KDE и множество

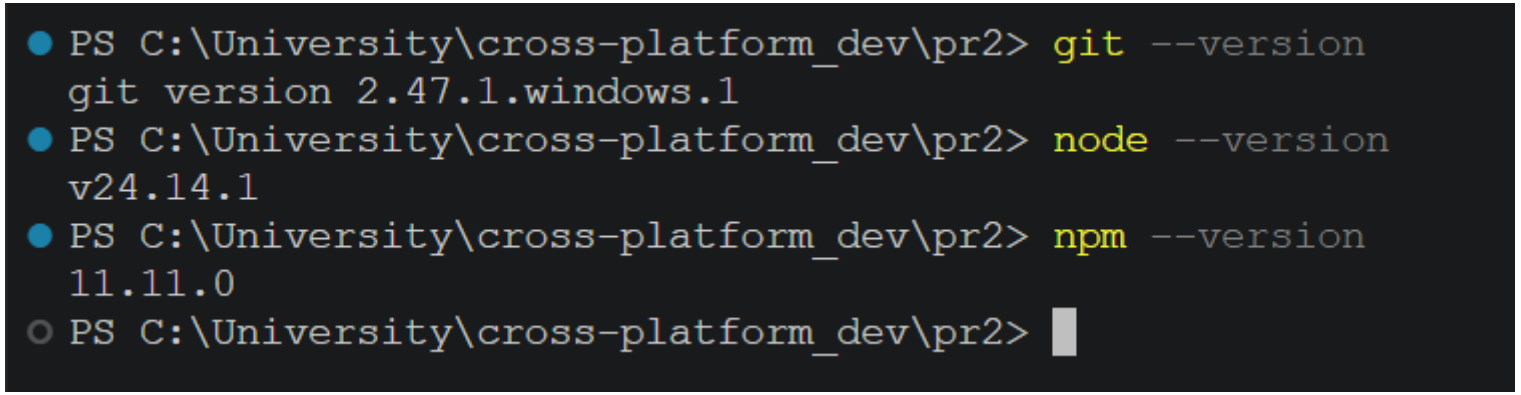
Категория	Инструмент	Разработчик	Язык	Целевые платформы	Механизм кросс-платформенности	Лицензия	Актуальная версия и дата	Пример применения
					файлов сборки из единого описания			C/C++ проектов
Десктопные UI-фреймворки	Qt 6	Qt Group / Qt Project	C++, QML	Windows, macOS, Linux, Android, iOS, embedded	Единый API и собственный кросс-платформенный UI-стек	LGPL/GPL или коммерческая	6.12 LTS / 30.09.2026	Telegram Desktop, OBS Studio (частично Qt)
Веб-технологии и WebView-обёртки	Electron	OpenJS Foundation / Electron	JavaScript/TypeScript	Windows, macOS, Linux	Chromium + Node.js поставляются вместе с приложением	MIT	45.0.0 / 20.10.2026	Visual Studio Code, Discord
Веб-технологии и WebView-обёртки	Tauri 2	Tauri Programme	Rust + JS/TS	Windows, macOS, Linux, iOS, Android	Системный WebView + ядро на Rust	Apache-2.0 / MIT	2.12.3 / 10.10.2026	Десктопные и мобильные WebView-приложения
Мобильные кросс-платформенные фреймворки	Flutter	Google	Dart	Android, iOS, Web, Windows, macOS, Linux	Единая кодовая база, собственный рендеринг, AOT/JIT	BSD-3-Clause	3.47.8 / 23.10.2026	Google Ads, eBay Motors
Мобильные кросс-платформенные фреймворки	Ionic + Capacitor	Ionic	JavaScript/TypeScript	Android, iOS, Web/PWA	Web UI + нативный контейнер и плагины доступа к API устройства	MIT	Ionic 9.0.9 / 21.10.2026; Capacitor 8.5.2 / 11.09.2026	Гибридные корпоративные и клиентские приложения
Виртуализация и контейнеризация	Docker Engine	Docker, Inc. / Moby	Go (реализация)	Windows, macOS, Linux	Контейнеры обеспечивают воспроизводимое окружение сборки и запуска	Apache-2.0 (Moby), коммерческие условия	29.9.2 / 20.10.2026	CI/CD, микросервисы, dev-окружения

Категория	Инструмент	Разработчик	Язык	Целевые платформы	Механизм кросс-платформенности	Лицензия	Актуальная версия и дата	Пример применения
						Docker Desktop		
Веб-технологии	PWA	W3C / WHATWG / разработчики браузеров	HTML, CSS, JavaScript	Современные браузеры на desktop/mobile	Стандарты Web App Manifest, Service Worker, Cache Storage и Web API	Зависит от приложения	Стандартный веб-стек; версия как у фреймворка отсутствует	Устанавливаемые веб-приложения, офлайн-сервисы

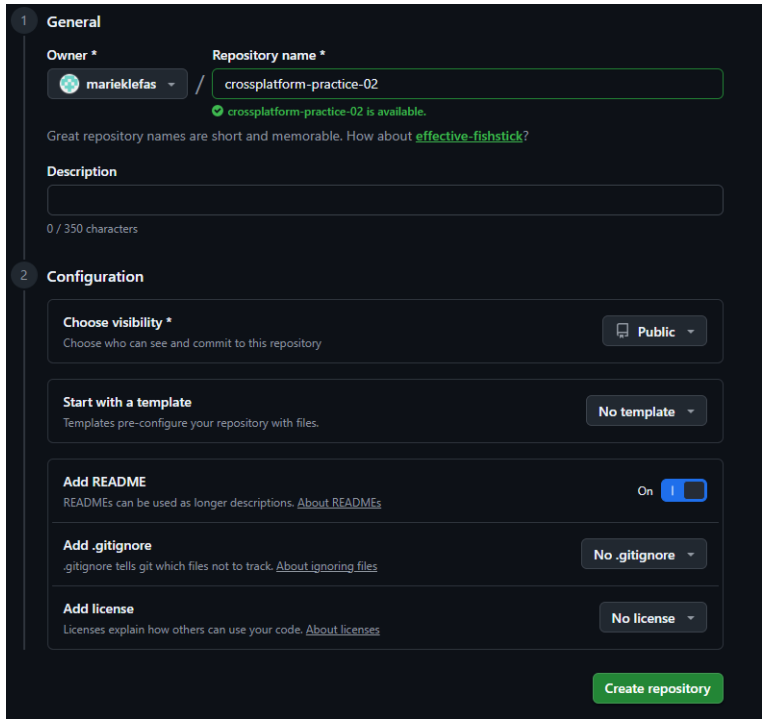
Вывод: современная кросс-платформенная разработка не сводится к одному подходу. Переносимость достигается интерпретацией, виртуальными машинами, переносимым исходным кодом, единым UI/API, WebView-обёртками либо контейнеризацией. Для мобильной разработки наиболее характерны Flutter и Ionic/Capacitor, для десктопа — Qt, Electron и Tauri, а Docker и CMake обеспечивают воспроизводимость и переносимость процесса сборки.

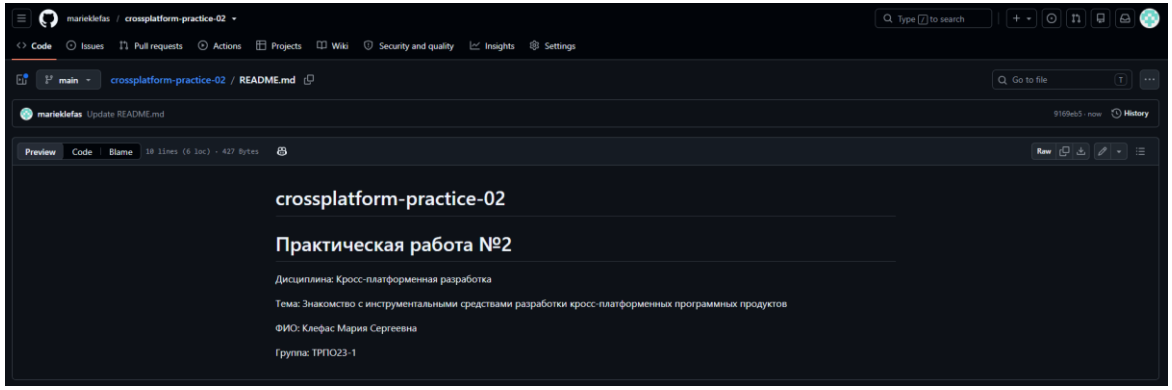
3. Задание 2. Подготовка рабочего окружения

3.1. Установка и проверка Visual Studio Code, Git и Node.js LTS



3.2. Создание удалённого репозитория





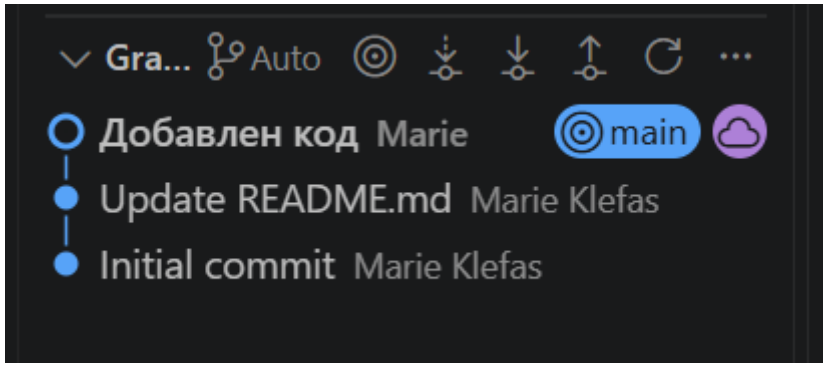
3.3. Клонирование репозитория и пробный коммит

Клонирование репозитория:

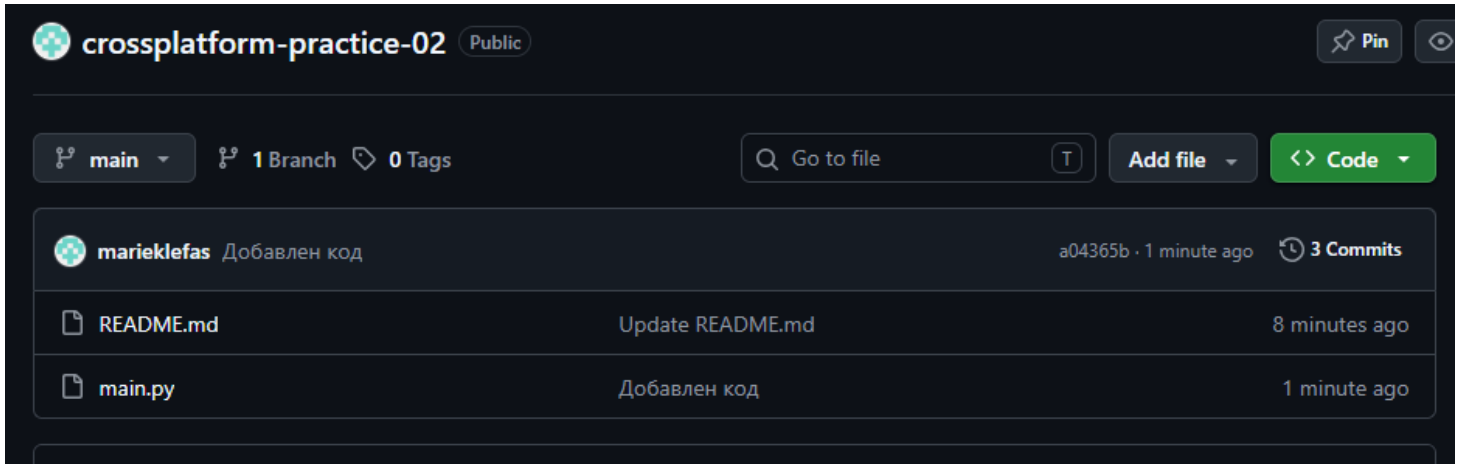
```
PS C:\University\cross-platform_dev\pr2> git clone https://github.com/marieklefas/crossplatform-practice-02.git

Cloning into 'crossplatform-practice-02'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
PS C:\University\cross-platform_dev\pr2>
```

Добавление нового файла (история коммитов):



На сайте гитхаба все тоже отразилось:



3.4. Результат задания 2

Рабочее окружение подготовленно (Visual Studio Code, Git и Node.js работают), создан публичный репозиторий crossplatform-practice-02, выполнено клонирование, создан один коммит и выполнен push.

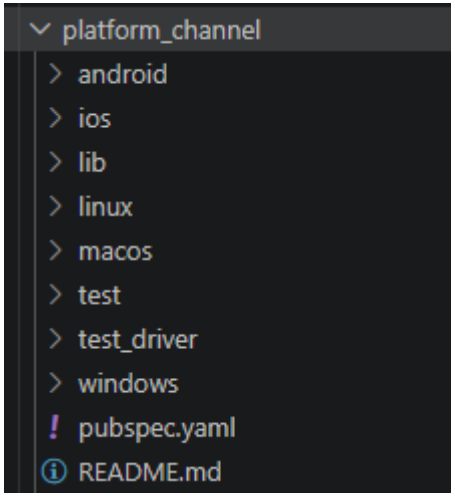
4. Задание 3. Знакомство с устройством кросс-платформенного проекта

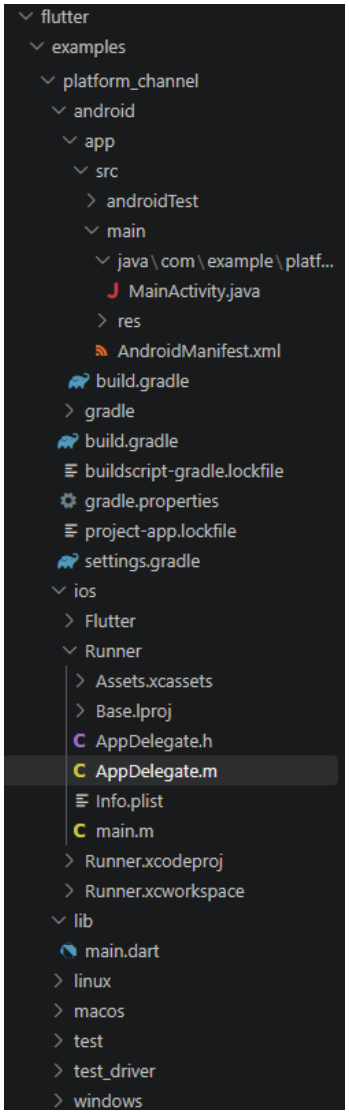
Для анализа выбран официальный пример Flutter «platform_channel» из репозитория github.com/flutter/ flutter. Проект демонстрирует обращение общей Dart-части приложения к платформенным API через каналы Flutter.

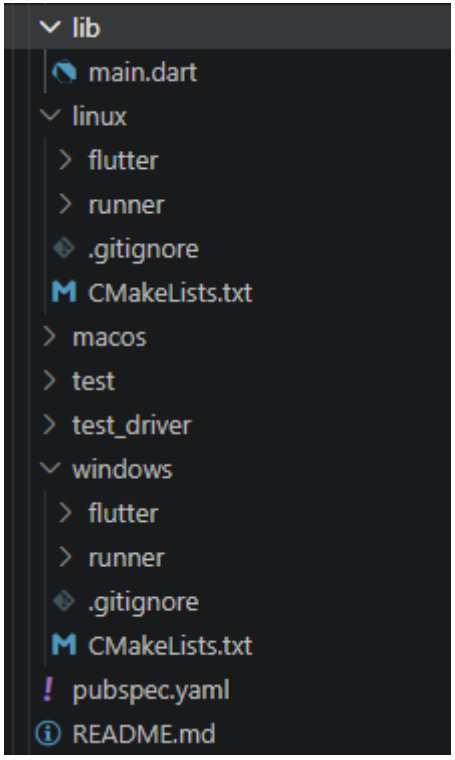
4.1. Скачиваем проект:

```
PS C:\University\cross-platform_dev\pr2> git clone --depth 1 https://github.com/flutter/flutter.git
Cloning into 'flutter'...
remote: Enumerating objects: 17597, done.
remote: Counting objects: 100% (17597/17597), done.
remote: Compressing objects: 100% (14247/14247), done.
remote: Total 17597 (delta 3715), reused 8690 (delta 2448), pack-reused 0 (from 0)
Receiving objects: 100% (17597/17597), 35.29 MiB | 15.55 MiB/s, done.
Resolving deltas: 100% (3715/3715), done.
Updating files: 100% (16284/16284), done.
PS C:\University\cross-platform_dev\pr2>
```

4.2. Дерево проекта:







4.3. Структура кросс-платформенного проекта

Таблица 2. Структура кросс-платформенного проекта

Каталог / файл	Назначение	Общая кодовая база или платформа
lib/main.dart	Основной Flutter-код: интерфейс, MethodChannel для запроса уровня батареи и EventChannel для статуса зарядки.	Общий код (Dart/Flutter)
pubspec.yaml	Конфигурация Flutter-проекта и зависимостей.	Общая конфигурация проекта
android/	Android-хост проекта, Gradle-файлы, AndroidManifest.xml и нативная реализация канала.	Android
android/app/src/main/AndroidManifest.xml	Манифест Android: параметры приложения, Activity и разрешения.	Android
android/.../MainActivity.java	Обрабатывает вызов getBatteryLevel и обращается к Android BatteryManager.	Android
ios/	Xcode-проект и нативная реализация iOS.	iOS
ios/Runner/AppDelegate.m	Создаёт FlutterMethodChannel/FlutterEventChannel, получает уровень батареи через UIDevice.	iOS
windows/	Runner и платформенный код Windows; обращение к WinAPI для получения статуса питания.	Windows
linux/	Платформенный Linux-код; доступ к системным средствам получения данных о питании.	Linux
README.md	Описание примера и способов сборки для поддерживаемых платформ.	Общая документация

4.4. Платформозависимые фрагменты

Первое место находится в общем Dart-коде lib/main.dart. В нём создаётся MethodChannel с именем канала и вызывается метод getBatteryLevel. Dart-код не знает, как именно операционная система получает заряд батареи: он только передаёт сообщение платформенной стороне.

static const MethodChannel methodChannel = MethodChannel('samples.flutter.io/battery');

final int? result = await methodChannel.invokeMethod('getBatteryLevel');

Второе место — нативная реализация на Android. Обработчик канала получает вызов getBatteryLevel и использует Android API BatteryManager. На iOS та же логическая операция реализована другим кодом через UIDevice. Следовательно, интерфейс вызова одинаков для общего кода, а фактическая реализация зависит от ОС.

В проекте также используется EventChannel для получения событий изменения состояния зарядки. Это ещё один пример разделения: Dart-подписчик общий, но источник событий реализуется отдельно на каждой платформе.

4.5. Вывод

Выбранный проект использует механизм кросс-платформенности Flutter: основная логика и пользовательский интерфейс пишутся на Dart в общей кодовой базе, а взаимодействие со специфическими возможностями операционной системы выполняется через платформенные каналы. Основной общий код расположен в каталоге lib/. Файлы android/, ios/, windows/ и linux/ содержат платформенные проекты и нативные реализации. В рассматриваемом примере доля общего прикладного кода высокая: интерфейс, состояние приложения и вызовы каналов находятся в одном Dart-файле. Платформозависимой остаётся реализация получения уровня батареи и событий зарядки. Flutter сериализует сообщение и передаёт его между Dart и нативным кодом, поэтому общий код не содержит прямых обращений к Android

BatteryManager или iOS UIDevice. Такой подход упрощает поддержку нескольких ОС, но каждая новая платформенная функция требует отдельной реализации для каждой целевой системы. При переносе на дополнительную платформу необходимо проверить наличие Flutter-рантайма и возможность реализовать требуемый канал. Для Astra Linux перенос в общем случае проще, поскольку Flutter поддерживает Linux, хотя системные API могут потребовать адаптации. Для ОС «Аврора» официальная поддержка Flutter ограничена, поэтому потребуются дополнительный порт, другой рантайм либо переработка клиентской части на поддерживаемом стеке. Главная граница переносимости в проекте проходит между lib/main.dart и каталогами платформенных хостов.

5. Задание 4. Сравнительный анализ инструментов — вариант 6

Вариант 6: PWA vs Ionic + Capacitor. Сценарий — сервис доставки еды с офлайн-режимом и push-уведомлениями; ограниченный бюджет, веб-команда.

Таблица 3. Сравнительный анализ PWA и Ionic + Capacitor

Критерий	PWA	Ionic + Capacitor
Целевые платформы и покрытие	Любые современные браузеры на Windows, macOS, Linux, Android и iOS. Установка возможна как веб-приложение; поддержка возможностей зависит от браузера.	Android, iOS и Web/PWA из общей веб-кодовой базы. Нативные проекты создаются отдельно для Android и iOS.
Отечественные ОС	Хороший вариант для Astra Linux и РЕД ОС при наличии современного браузера. Для «Авроры» зависит от доступного браузера/Web API.	Web-версия работает там же, где PWA. Нативный Capacitor официально ориентирован на Android/iOS; прямой поддержки «Авроры» нет.
Язык и порог входа	HTML, CSS, JavaScript/TypeScript. Минимальный порог для веб-команды.	Тот же веб-стек; Ionic добавляет UI-компоненты, Capacitor — native runtime. Для сложных плагинов нужны базовые знания Xcode/Android Studio.

Критерий	PWA	Ionic + Capacitor
Рендеринг и производительность	Рендеринг браузером. Для сервиса доставки обычно достаточен; зависит от браузера и качества веб-приложения.	UI также работает через WebView, но приложение упаковано как нативное. Для большинства CRUD/контентных сценариев производительность достаточна.
Офлайн-режим	Service Worker может перехватывать запросы и отдавать данные из Cache Storage; возможно построение полноценного offline-first режима.	Можно использовать те же веб-механизмы, а также нативные плагины/хранилища. Удобнее контролировать локальные данные на мобильных ОС.
Push-уведомления	Возможны через Web Push там, где браузер и ОС это поддерживают; поведение и ограничения различаются между платформами.	Официальный Capacitor Push Notifications plugin даёт доступ к нативным push. Android использует FCM; на iOS включается Push Notifications capability.
Размер дистрибутива и ресурсы	Минимальный: отдельный APK/IPA не обязателен; приложение загружается как веб-ресурс и может устанавливаться из браузера.	Больше PWA из-за нативной оболочки и проектов Android/iOS, но заметно легче решений со встроенным Chromium.
Экосистема и зрелость	Основана на веб-стандартах, Service Worker, Web App Manifest и широком JS-экосистеме.	Ionic — зрелый UI-фреймворк; Capacitor — современная замена Cordova, имеет официальные и community-плагины.
Актуальность	PWA — не единый продукт с номером версии; развивается вместе со стандартами и браузерами.	Ionic 9.0.9 (21.10.2026); стабильная ветка Capacitor 8.5.2 (11.09.2026).
Лицензия и стоимость	Веб-стандарты бесплатны; расходы — хостинг, backend и инфраструктура push.	Ionic Framework и Capacitor имеют MIT-лицензию. Базовая разработка бесплатна; отдельные облачные/enterprise-сервисы необязательны.
Распространение	Публикация по URL и установка из браузера; нет обязательной модерации магазина. Возможность попадания в app stores зависит от дополнительной упаковки.	Нативные сборки можно распространять через Google Play, App Store, RuStore (для Android) и прямую загрузку там, где это допустимо.

Вывод:

Для заданного сервиса доставки еды лучше выбрать Ionic + Capacitor. PWA является самым дешёвым и быстрым вариантом для веб-команды: оно использует привычные HTML, CSS и JavaScript/TypeScript, не требует отдельной нативной кодовой базы и поддерживает офлайн-режим через Service Worker и Cache Storage. Однако в данном сценарии push-уведомления относятся к ключевым функциям: пользователю нужно сообщать о принятии заказа, смене статуса и прибытии курьера. Здесь Capacitor даёт более предсказуемый мобильный путь, поскольку официальный Push Notifications plugin обращается к нативным механизмам уведомлений; на Android используется Firebase Cloud Messaging, а на iOS включается системная Push Notifications capability.

Второй аргумент — каналы распространения. Capacitor создаёт полноценные Android- и iOS-проекты, поэтому приложение можно опубликовать в Google Play, App Store и, для Android-сборки, RuStore, сохранив при этом общую веб-кодовую базу.

Третий аргумент — доступ к функциям устройства. Плагины Capacitor предоставляют единый JavaScript API поверх нативных API и позволяют при необходимости написать собственный Swift/Kotlin-плагин. Для ограниченного бюджета это хороший компромисс: команда остаётся в знакомом веб-стеке, но получает более надёжный доступ к мобильным возможностям. PWA целесообразно сохранить как веб-версию того же продукта: Capacitor официально поддерживает Web/PWA наряду с Android и iOS.

Таким образом, архитектура может быть web-first, а Capacitor использоваться для мобильной упаковки и нативных возможностей.

6. Ответы на контрольные вопросы

1. Дайте определение кросс-платформенного программного обеспечения. Перечислите и охарактеризуйте уровни совместимости программ.

Кросс-платформенное ПО способно работать более чем на одной аппаратной или программной платформе. Основные уровни совместимости: машинный код, байт-код, исходный код и API.

2. Чем байт-код JVM отличается от WebAssembly как средство обеспечения переносимости?

JVM-байт-код ориентирован на виртуальную машину Java и экосистему JVM-языков. WebAssembly — переносимый низкоуровневый байт-код, изначально ориентированный на браузеры, но применимый и вне браузера через Wasm-runtime.

3. Как устроен рендеринг во Flutter и почему этому фреймворку не требуются нативные компоненты ОС?

Flutter обычно не собирает интерфейс из нативных UI-компонентов ОС. Фреймворк формирует собственное дерево виджетов и рендерит интерфейс своим графическим стеком, поэтому внешний вид и поведение легче унифицировать между платформами.

4. Что изменилось в React Native с переходом на «новую архитектуру» (версии 0.76–0.82)?

Новая архитектура использует Fabric, TurboModules и JSI вместо старого асинхронного bridge. Это уменьшает накладные расходы обмена между JavaScript и нативной частью и улучшает интеграцию с современным React.

5. В чём разница между Kotlin Multiplatform и Compose Multiplatform? Что изменил стабильный статус iOS-версии CMP 1.8?

Kotlin Multiplatform предназначен прежде всего для совместного использования кода и бизнес-логики между платформами. Compose Multiplatform добавляет общий декларативный UI. Стабильная поддержка iOS сделала общий UI практичнее для production-сценариев.

6. Почему приложения Electron имеют большой размер дистрибутива и как эту проблему решает Tauri?

Electron поставляет с каждым приложением Chromium и Node.js, что увеличивает размер и потребление памяти. Tauri использует системный WebView, а нативную часть реализует на Rust, поэтому дистрибутив обычно значительно меньше.

7. Какие инструменты подходят для разработки под ОС «Аврора» и почему?

Основной нативный стек «Авроры» связан с Qt/QML и «Аврора SDK». Поэтому Qt является наиболее прямым вариантом. Для других технологий может потребоваться слой совместимости или перенос клиентской части.

8. Назовите особенности публикации приложения в RuStore по сравнению с Google Play.

RuStore ориентирован прежде всего на Android-приложения и поддерживает APK/AAB. Требуются учётная запись разработчика и модерация. В методических указаниях также отмечено, что публикация бесплатна.

9. В каких случаях нативная разработка предпочтительнее кросс-платформенной? Приведите два примера.

Когда критичны максимальная производительность, глубокая интеграция с платформой или немедленный доступ к новым системным API. Примеры: высокопроизводительная камера/AR и приложение с большим количеством специальных фоновых системных функций.

10. Какую роль играют CMake и стандарт POSIX в обеспечении переносимости на уровне исходного кода?

POSIX стандартизует системные интерфейсы, уменьшая зависимость исходного кода от конкретной Unix-подобной ОС. CMake описывает сборку платформонезависимо и генерирует файлы сборочной системы для каждой целевой платформы.

11. Почему Apache Cordova сегодня не рекомендуется для новых проектов и что пришло ей на смену?

Для новых проектов Cordova считается устаревшим подходом. Современной заменой в экосистеме Ionic является Capacitor, который лучше интегрируется с современным веб-стеком и нативными проектами.

12. Как ИИ-ассистенты разработчика влияют на выбор инструментария в 2025–2026 годах? Приведите аргументы «за» и «против» их использования в учебном проекте.

Они ускоряют написание шаблонного кода, поиск ошибок и миграцию между фреймворками. Плюсы — скорость и снижение рутины; минусы — возможность генерации неверного или устаревшего кода, необходимость проверки и риск непонимания архитектуры.

7. Использованные источники

1. Методические указания к практической работе № 2 «Знакомство с инструментальными средствами разработки кросс-платформенных программных продуктов».

2. Python Releases — https://www.python.org/downloads/ (дата обращения: 26.10.2026).

3. Node.js Releases — https://nodejs.org/en/blog/release (дата обращения: 26.10.2026).

4. CMake Releases — https://cmake.org/download/ и CMake Discourse Announcements (дата обращения: 26.10.2026).

5. Qt Releases — https://www.qt.io/development/qt-framework/latest-releases (дата обращения: 26.10.2026).

6. Electron Releases — https://releases.electronjs.org/ (дата обращения: 26.10.2026).

7. Tauri Releases — https://tauri.app/release/ (дата обращения: 26.10.2026).

8. Flutter Documentation / SDK Archive — https://docs.flutter.dev/install/archive (дата обращения: 26.10.2026).

9. Ionic Release Notes — https://ionicframework.com/docs/reference/release-notes (дата обращения: 26.10.2026).

10. Capacitor Documentation — https://capacitorjs.com/docs (дата обращения: 26.10.2026).

11. Docker Engine Release Notes — https://docs.docker.com/engine/release-notes/ (дата обращения: 26.10.2026).

12. Flutter platform_channel example — https://github.com/flutter/flutter/tree/master/examples/platform_channel (дата обращения: 26.10.2026).

13. web.dev, Progressive Web Apps / Service Workers — https://web.dev/learn/pwa/ (дата обращения: 26.10.2026).